

LAPORAN PRAKTIKUM
ALGORITMA PEMROGRAMAN 2
MODUL 14 - SORTING



NAMA : AIS NURHANIFAH

NIM : 109082500162

KELAS : S1IF-13-06

PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS INFORMATIKA
TELKOM UNIVERSITY PURWOKERTO
2026

A. Dasar Teori

Pengurutan data atau *sorting* merupakan salah satu konsep fundamental dalam algoritma pemrograman yang bertujuan untuk mengatur sekumpulan elemen data ke dalam urutan tertentu, baik secara menaik (*ascending*) maupun menurun (*descending*). Proses pengurutan ini sangat krusial dalam dunia informatika karena data yang terstruktur dengan rapi akan jauh lebih mudah dikelola, dianalisis, dan efisien saat diproses oleh algoritma pencarian. Dalam bahasa pemrograman Go, manipulasi data array atau slice untuk keperluan *sorting* sering kali memanfaatkan parameter berbasis *pointer*. Melalui mekanisme *pointer*, fungsi pengurutan dapat langsung memodifikasi susunan elemen pada alamat memori array asli di fungsi utama (*main*) tanpa perlu membuat replika data baru atau mengembalikan nilai bertipe array secara eksplisit.

Salah satu algoritma pengurutan sederhana yang sering dipelajari adalah *Selection Sort*. Logika dasar dari *Selection Sort* berfokus pada pencarian nilai ekstrem—baik nilai minimum untuk pengurutan membesar atau nilai maksimum untuk pengurutan mengecil—di dalam sebuah array. Algoritma ini bekerja dengan menggunakan dua buah perulangan bersarang (*nested loop*). Perulangan luar bertindak sebagai penanda posisi indeks yang sedang diproses, sementara perulangan dalam bertugas menelusuri sisa elemen array guna menemukan nilai paling ekstrem. Setelah penelusuran selesai dan indeks elemen ekstrem (*idx_min* atau *idx_max*) teridentifikasi, program akan melakukan optimasi pengecekan untuk menukar posisi elemen tersebut dengan elemen di posisi indeks perulangan luar apabila posisinya belum sesuai.

Algoritma pengurutan mendasar lainnya adalah *Insertion Sort* yang memiliki karakteristik menyerupai cara manusia menyusun kartu remi di tangan. Cara kerja *Insertion Sort* dimulai dari elemen kedua, di mana elemen tersebut diambil sebagai kunci (*key* atau *temp*) untuk dibandingkan dengan elemen-elemen di sebelah kirinya yang sudah dianggap terurut. Jika kondisi pengurutan terpenuhi—misalnya elemen sebelumnya lebih besar dari kunci pada pengurutan *ascending*—maka elemen yang lebih besar tersebut akan digeser ke posisi kanan satu demi satu. Proses pergeseran elemen ini terus berlangsung ke arah kiri menggunakan perulangan dalam hingga program menemukan posisi selipan yang tepat bagi elemen kunci tersebut.

Penerapan fungsi pengurutan dalam praktiknya tidak hanya terbatas pada tipe data primitif seperti *integer* atau *string*, melainkan juga dapat diimplementasikan pada tipe data bentukan seperti *struct*. Di dalam bahasa pemrograman Go, pengurutan objek *array of struct* dilakukan dengan menjadikan salah satu komponen field di dalam *struct* tersebut sebagai kunci atau acuan pembanding utama, contohnya berdasarkan nilai IPK mahasiswa atau rating sebuah buku. Penggunaan struktur data ini memberikan keuntungan besar bagi konsistensi data karena ketika operasi pertukaran (*swap*) dieksekusi, algoritma akan menukar seluruh record atau entitas *struct* secara utuh. Dengan demikian, seluruh field informasi lainnya yang terikat di dalam objek *struct* tersebut akan ikut berpindah tempat secara otomatis tanpa merusak validitas data.

B. Guided

1. [selSortArray.go]

```
package main

import "fmt"

func selecetionSortArray(angka *[5]int){
    var idx_min, i, j int
    for i = 0; i < len(angka) - 1; i++ { //loop luar (iterasi
    sortingnya)
        idx_min = i
        for j = i + 1; j < len(angka); j++ { //loop dalam (mencari
    nilai ekstrim)
            if angka[j] <= angka[idx_min] { //sorting terkecil ke
    terbesar
                idx_min = j
            }
        }
        if idx_min != i {
            angka[i], angka[idx_min] = angka[idx_min], angka[i]
        }
    }
}

func main(){
    var arrAngka [5]int

    for i := 0; i < len(arrAngka); i++ {
        fmt.Printf("Masukkan data angka ke-%d : ", i)
        fmt.Scan(&arrAngka[i])
    }
    fmt.Println()

    fmt.Println("=== SEBELUM DISORTING ===")
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }

    fmt.Println("\n=== SETELAH DISORITNG ===")
    selecetionSortArray(&arrAngka)
    for i := 0; i < len(arrAngka); i++ {
        fmt.Print(arrAngka[i], " - ")
    }
}
```

Output :

```
PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 12 - Modul 14\guided1.g
Masukkan data angka ke-0 : 35
Masukkan data angka ke-1 : 12
Masukkan data angka ke-2 : 85
Masukkan data angka ke-3 : 24
Masukkan data angka ke-4 : 5

=== SEBELUM DISORTING ===
35 - 12 - 85 - 24 - 5 -
=== SETELAH DISORTING ===
5 - 12 - 24 - 35 - 85 -
PS D:\Praktikum\109082500162_Ais-Nurhanifah> 
```

Penjelasan :

Program ini adalah implementasi **selection sort** untuk mengurutkan array berisi 5 bilangan bulat dari terkecil ke terbesar. Program terdiri dari dua bagian utama yaitu fungsi `selectionSortArray` dan fungsi `main`. Yang menarik dari program ini adalah penggunaan **pointer** `*[5]int` sebagai parameter fungsi, sehingga perubahan yang dilakukan di dalam fungsi langsung mempengaruhi array asli di `main` tanpa perlu mengembalikan nilai apapun.

Algoritma **selection sort** bekerja dengan dua perulangan bersarang. Perulangan luar (`i`) berjalan dari indeks 0 hingga `n-2`, menandai posisi yang akan diisi oleh nilai minimum. Perulangan dalam (`j`) berjalan dari `i+1` hingga akhir array untuk mencari indeks nilai terkecil (`idx_min`). Setelah nilai terkecil ditemukan, jika `idx_min != i` maka dilakukan pertukaran posisi. Pengecekan `idx_min != i` ini merupakan optimasi kecil agar pertukaran tidak dilakukan jika elemen sudah berada di posisi yang benar, sehingga menghindari operasi swap yang tidak perlu.

2. [selSortStruct.go]

```
package main

import "fmt"

type mahasiswa struct {
    nama, nim string
    IPK        float64
}

func SelectionSortArray(smhs *[5]mahasiswa) {
    var idx_min, i, j int
    for i = 0; i < len(smhs)-1; i++ { //Loop Luar
        idx_min = i
        for j = i + 1; j < len(smhs); j++ { //Loop dalam mencari
            nilai ekstrim
            if smhs[j].IPK < smhs[idx_min].IPK { //sorting terkecil
                ke terbesar
            }
        }
    }
}
```

```

        idx_min = j
    }
}
if idx_min != i {
    sMHS[i], sMHS[idx_min] = sMHS[idx_min], sMHS[i]
}
}
}

func main() {
    var structMHS [5]mahasiswa

    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Masukkan Data Mahasiswa Ke-%d ---\n", i)
        fmt.Print("Nama : ")
        fmt.Scan(&structMHS[i].nama)
        fmt.Print("NIM : ")
        fmt.Scan(&structMHS[i].nim)
        fmt.Print("IPK : ")
        fmt.Scan(&structMHS[i].IPK)
    }
    fmt.Println()

    fmt.Println("=== SEBELUM SORITNG === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()

    SelectionSortArray(&structMHS)
    fmt.Println("=== SETELAH SORITNG === ")
    for i := 0; i < len(structMHS); i++ {
        fmt.Printf("--- Data Mahasiswa Ke-%d ---\n", i)
        fmt.Printf("Nama : %s\n", structMHS[i].nama)
        fmt.Printf("NIM : %s\n", structMHS[i].nim)
        fmt.Printf("IPK : %.2f\n", structMHS[i].IPK)
    }
    fmt.Println("=====")
    fmt.Println()
}

```

Output :

```
PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 12 - Modul 14\guided2.go"
--- Masukkan Data Mahasiswa Ke-0 ---
Nama : ais
NIM : 101101
IPK : 4.0
--- Masukkan Data Mahasiswa Ke-1 ---
Nama : nur
NIM : 101102
IPK : 3.8
--- Masukkan Data Mahasiswa Ke-2 ---
Nama : hani
NIM : 101103
IPK : 3.6
--- Masukkan Data Mahasiswa Ke-3 ---
Nama : ifah
NIM : 101104
IPK : 3.3
--- Masukkan Data Mahasiswa Ke-4 ---
Nama : afif
NIM : 101105
IPK : 3.0

=== SEBELUM SORTING ===
--- Data Mahasiswa Ke-0 ---
Nama : ais
NIM : 101101
IPK : 4.00
--- Data Mahasiswa Ke-1 ---
Nama : nur
```

```
=== SEBELUM SORTING ===
--- Data Mahasiswa Ke-0 ---
Nama : ais
NIM : 101101
IPK : 4.00
--- Data Mahasiswa Ke-1 ---
Nama : nur
NIM : 101102
IPK : 3.80
--- Data Mahasiswa Ke-2 ---
Nama : hani
NIM : 101103
IPK : 3.60
--- Data Mahasiswa Ke-3 ---
Nama : ifah
NIM : 101104
IPK : 3.30
--- Data Mahasiswa Ke-4 ---
Nama : afif
NIM : 101105
IPK : 3.00
=====
```

```
=== SETELAH SORTING ===
--- Data Mahasiswa Ke-0 ---
Nama : afif
NIM : 101105
IPK : 3.00
--- Data Mahasiswa Ke-1 ---
Nama : ifah
NIM : 101104
IPK : 3.30
--- Data Mahasiswa Ke-2 ---
Nama : hani
NIM : 101103
IPK : 3.60
--- Data Mahasiswa Ke-3 ---
Nama : nur
NIM : 101102
IPK : 3.80
--- Data Mahasiswa Ke-4 ---
Nama : ais
NIM : 101101
IPK : 4.00
=====

PS D:\Praktikum\109082500162_Ais-Nurhanifah> |
```

Penjelasan :

Program ini adalah pengembangan dari program selection sort sebelumnya, namun kali ini mengurutkan **array of struct** bukan array integer biasa. Struct mahasiswa memiliki tiga field yaitu nama dan nim bertipe string serta IPK bertipe float64. Yang menjadi **kunci pengurutan** adalah field IPK, di mana data diurutkan dari IPK terkecil ke terbesar. Penggunaan struct memungkinkan program menyimpan beberapa informasi berbeda dalam satu kesatuan data, sehingga ketika satu elemen struct ditukar posisinya, semua field (nama, nim, IPK) ikut berpindah bersama-sama secara otomatis.

Sama seperti sebelumnya, fungsi SelectionSortArray menerima **pointer** `*[5]mahasiswa` agar perubahan urutan yang dilakukan di dalam fungsi langsung mempengaruhi array asli di main. Perulangan luar berjalan dari indeks 0 hingga `n-2` untuk menentukan posisi yang akan diisi, sedangkan perulangan dalam mencari indeks mahasiswa dengan IPK terkecil (`idx_min`) dari posisi `i+1` hingga akhir array. Jika ditemukan IPK yang lebih kecil maka `idx_min` diperbarui, dan setelah perulangan dalam selesai dilakukan pertukaran seluruh struct jika `idx_min != i`. Karena yang ditukar adalah **seluruh struct**, bukan hanya IPK-nya saja, maka data nama dan NIM tetap konsisten mengikuti IPK masing-masing mahasiswa.

3. [insertionSort.go]

```
package main

import "fmt"

const nMax int = 4321

type arrInt [nMax]int

func insertionSort(T *arrInt, n int) {
    /* I.S. terdefinisi array T yang berisi n bilangan bulat
       F.S. array T terurut secara mengecil atau descending dengan
       INSERTION SORT */

    var temp, i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]
```

```

        for j > 0 && temp > T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrInt
    var n int

    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSort(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 12 - Modul 14\tempCodeRun
File.go"
4
12
45
8
23
45 23 12 8
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program ini mengimplementasikan algoritma **insertion sort** untuk mengurutkan array bilangan bulat secara **descending (mengecil/terbesar ke terkecil)**. Program menggunakan tipe data custom `arrInt` yaitu array berukuran `nMax = 4321` elemen bertipe `int`. Input pertama adalah `n` yaitu banyaknya data, kemudian diikuti `n` bilangan bulat yang akan diurutkan. Fungsi `insertionSort` menerima **pointer** `*arrInt` agar perubahan langsung mempengaruhi array asli, serta parameter `n` untuk mengetahui batas data yang valid di dalam array.

Perbedaan utama program ini dengan insertion sort ascending adalah pada kondisi perulangan dalam, yaitu $\text{temp} > T[j-1]$. Kondisi ini berarti elemen temp akan terus digeser ke kiri **selama temp lebih besar** dari elemen sebelumnya, sehingga elemen-elemen yang lebih kecil digeser ke kanan dan nilai besar menggelembung ke depan array. Hasilnya adalah array terurut dari nilai terbesar di indeks 0 hingga terkecil di indeks $n-1$. Selain itu perulangan pada program ini menggunakan gaya **Pascal-style** (while loop dengan $i = i + 1$ dan $j = j - 1$) bukan gaya Go/C biasa dengan for $i++$, namun hasil dan logikanya tetap sama persis.

4. [insertionSortString.go]

```
package main

import "fmt"

const nMax int = 100

type arrString [nMax]string

func insertionSortString(T *arrString, n int) {
    var temp string
    var i, j int

    i = 1
    for i <= n-1 {
        j = i
        temp = T[j]

        for j > 0 && temp < T[j-1] {
            T[j] = T[j-1]
            j = j - 1
        }

        T[j] = temp
        i = i + 1
    }
}

func main() {
    var data arrString
    var n int

    fmt.Scan(&n)
```

```

    for i := 0; i < n; i++ {
        fmt.Scan(&data[i])
    }

    insertionSortString(&data, n)

    for i := 0; i < n; i++ {
        fmt.Print(data[i], " ")
    }
}

```

Output :

```

PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 12 -
Modul 14\guided4.go"
5 Jeruk Apel Mangga Duku Belimbing
Apel Belimbing Duku Jeruk Mangga
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program ini mengimplementasikan algoritma **insertion sort** untuk mengurutkan array berisi **string** secara **ascending (alfabetis A-Z)**. Program menggunakan tipe data custom `arrString` yaitu array berukuran `nMax = 100` elemen bertipe `string`. Input pertama adalah `n` yaitu banyaknya data string, kemudian diikuti `n` kata yang akan diurutkan secara alfabetis. Yang menarik dari program ini adalah perbandingan string di Go menggunakan operator `<` dan `>` biasa, di mana Go secara otomatis membandingkan string berdasarkan nilai **ASCII** karakter per karakter dari kiri ke kanan, sehingga `"apel" < "mangga"` bernilai `true` karena huruf `a` memiliki nilai ASCII lebih kecil dari `m`.

Kondisi perulangan dalam menggunakan `temp < T[j-1]`, yang berarti elemen `temp` akan terus digeser ke kiri **selama temp lebih kecil secara alfabetis** dari elemen sebelumnya. Ini menghasilkan urutan **ascending alfabetis** karena string yang lebih kecil (lebih awal secara alfabet) akan bergerak ke depan array. Proses ini identik dengan cara manusia mengurutkan kata secara alfabet, yaitu membandingkan huruf pertama, jika sama lanjut ke huruf kedua, dan seterusnya. Fungsi menerima pointer `*arrString` agar perubahan langsung mempengaruhi array asli tanpa perlu nilai kembalian.

C. Unguided

1. Soal Unguided 1

Hercules, preman terkenal seantero ibukota, memiliki kerabat di banyak daerah. Tentunya Hercules sangat suka mengunjungi semua kerabatnya itu. Diberikan masukan nomor rumah dari semua kerabatnya di suatu daerah, buatlah program rumahkerabat yang akan menyusun nomor-nomor rumah kerabatnya secara terurut membesar menggunakan algoritma selection sort.

Masukan dimulai dengan sebuah integer n ($0 < n < 1000$), banyaknya daerah kerabat Hercules tinggal. Isi n baris berikutnya selalu dimulai dengan sebuah integer m ($0 < m < 1000000$) yang menyatakan banyaknya rumah kerabat di daerah tersebut, diikuti dengan rangkaian bilangan bulat positif, nomor rumah para kerabat.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar dimasing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 2 7 9 13 15 27 39 75 133 189 1 4 9

Keterangan: Terdapat 3 daerah dalam contoh input, dan di masing-masing daerah mempunyai 5, 6, dan 3 kerabat.

Jawaban :

```
package main

import "fmt"

func selectionSort(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        minIdx := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[minIdx] {
                minIdx = j
            }
        }
        arr[i], arr[minIdx] = arr[minIdx], arr[i]
    }
}
```

```

func main() {
    var n int
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        var m int
        fmt.Scan(&m)

        rumah := make([]int, m)
        for j := 0; j < m; j++ {
            fmt.Scan(&rumah[j])
        }

        selectionSort(rumah)

        for j := 0; j < m; j++ {
            if j > 0 {
                fmt.Print(" ")
            }
            fmt.Print(rumah[j])
        }
        fmt.Println()
    }
}

```

Output :

```

PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 12 - Modul 14\unguided1
o"
3
5 2 1 7 9 13
1 2 7 9 13
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 4 9
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program **rumahkerabat** adalah program yang bertugas menyusun nomor-nomor rumah kerabat Hercules secara terurut membesar (ascending) menggunakan algoritma **selection sort**. Input program dimulai dengan sebuah integer n yang menyatakan banyaknya daerah kerabat Hercules, kemudian untuk setiap daerah dibaca integer m yang menyatakan banyaknya kerabat di daerah tersebut, diikuti dengan m buah nomor rumah. Program kemudian mengurutkan nomor-nomor rumah tersebut dan mencetak hasilnya per daerah dalam satu baris, dipisahkan spasi.

Algoritma **selection sort** bekerja dengan cara mencari elemen terkecil dari posisi i hingga akhir array, lalu menukarnya dengan elemen pada posisi i , kemudian nilai i bertambah satu dan proses diulang hingga semua elemen terurut. Misalnya untuk array {2, 1, 7, 9, 13}, pada iterasi pertama dicari nilai terkecil dari seluruh array yaitu 1, lalu ditukar dengan elemen pertama 2 sehingga menjadi {1, 2, 7, 9, 13}. Kompleksitas waktu selection sort adalah $O(n^2)$ karena terdapat dua perulangan bersarang, namun tetap efektif untuk jumlah data yang wajar seperti pada soal ini.

2. Soal Unguided 2

Belakangan diketahui ternyata Hercules itu tidak berani menyeberang jalan, maka selalu diusahakan agar hanya menyeberang jalan sesedikit mungkin, hanya diujung jalan. Karena nomor rumah sisi kiri jalan selalu ganjil dan sisi kanan jalan selalu genap, maka buatlah program kerabat dekat yang akan menampilkan nomor rumahmulai dari nomor yang ganjil lebih dulu terurut membesar dan kemudian menampilkan nomor rumah dengan nomor genap terurut mengecil.

Format **Masukan** masih persis sama seperti sebelumnya.

Keluaran terdiri dari n baris, yaitu rangkaian rumah kerabatnya terurut membesar untuk nomor ganjil, diikuti dengan terurut mengecil untuk nomor genap, dimasing-masing daerah.

No	Masukan	Keluaran
1	3 5 2 1 7 9 13 6 189 15 27 39 75 133 3 4 9 1	1 13 12 8 2 15 27 39 75 133 189 8 4 2

Keterangan: Terdapat 3 daerah dalam contoh masukan. Baris kedua berisi campuran bilangan ganjil dan genap. Baris berikutnya hanya berisi bilangan ganjil, dan baris terakhir hanya berisi bilangan genap.

Petunjuk:

- Waktu pembacaan data, bilangan ganjil dan genap dipisahkan ke dalam dua array yang berbeda, untuk kemudian masing-masing diurutkan tersendiri.
- Atau, tetap disimpan dalam satu array, diurutkan secara keseluruhan. Tetapi pada waktu pencetakan, mulai dengan mencetak semua nilai ganjil lebih dulu, kemudian setelah selesai cetaklah semua nilai genapnya.

Jawaban :

```
package main

import "fmt"

// Fungsi untuk mengurutkan array ganjil secara membesar (Ascending)
func selectionSortAsc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        idxMin := i
        for j := i + 1; j < n; j++ {
            if arr[j] < arr[idxMin] {
                idxMin = j
            }
        }
        if idxMin != i {
            arr[i], arr[idxMin] = arr[idxMin], arr[i]
        }
    }
}

// Fungsi untuk mengurutkan array genap secara mengecil (Descending)
func selectionSortDesc(arr []int) {
    n := len(arr)
    for i := 0; i < n-1; i++ {
        idxMax := i
        for j := i + 1; j < n; j++ {
            if arr[j] > arr[idxMax] {
                idxMax = j
            }
        }
        if idxMax != i {
            arr[i], arr[idxMax] = arr[idxMax], arr[i]
        }
    }
}

func main() {
    var t, n, nomor int

    // Membaca jumlah daerah (test case)
    fmt.Scan(&t)

    // Loop sebanyak jumlah daerah
    for i := 0; i < t; i++ {
```

```

// Membaca jumlah rumah di daerah tersebut
fmt.Scan(&n)

var ganjil []int
var genap []int

// Membaca nomor rumah dan langsung memisahnya
for j := 0; j < n; j++ {
    fmt.Scan(&nomor)
    if nomor%2 != 0 {
        ganjil = append(ganjil, nomor)
    } else {
        genap = append(genap, nomor)
    }
}

// Urutkan ganjil (Ascending) dan genap (Descending)
selectionSortAsc(ganjil)
selectionSortDesc(genap)

// Cetak hasil: ganjil dulu baru genap
for _, v := range ganjil {
    fmt.Print(v, " ")
}
for _, v := range genap {
    fmt.Print(v, " ")
}
fmt.Println() // Baris baru untuk daerah berikutnya
}

```

Output :

```

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 12 - Modul 14\unguided..."
5 2 1 7 9 13
1 7 9 13 2
6 189 15 27 39 75 133
15 27 39 75 133 189
3 4 9 1
1 9 4
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program **kerabat dekat** adalah pengembangan dari program sebelumnya yang juga mengelola nomor rumah kerabat Hercules, namun dengan aturan pengurutan yang berbeda. Latar belakangnya adalah Hercules yang tidak berani menyeberang jalan, di mana sisi kiri jalan selalu bernomor ganjil dan sisi kanan

selalu bernomor genap. Program ini memisahkan nomor rumah ganjil dan genap, lalu mengurutkan nomor ganjil secara **membesar (ascending)** dan nomor genap secara **mengecil (descending)**, kemudian mencetak nomor ganjil terlebih dahulu baru nomor genap dalam satu baris per daerah.

Algoritma yang digunakan tetap **selection sort**, namun diterapkan dua kali dengan arah berbeda. Untuk nomor ganjil digunakan selection sort ascending yaitu mencari nilai **terkecil** lalu diletakkan di depan, sedangkan untuk nomor genap digunakan selection sort descending yaitu mencari nilai **terbesar** lalu diletakkan di depan. Proses pemisahan dilakukan saat membaca input, yaitu jika $x \% 2 \neq 0$ maka masuk slice ganjil, selain itu masuk slice genap. Setelah keduanya diurutkan, hasilnya digabung dan dicetak dalam satu baris per daerah.

3. Soal Unguided 3

Buatlah sebuah program yang digunakan untuk membaca data integer seperti contoh yang diberikan di bawah ini, kemudian diurutkan (menggunakan metoda insertion sort), dan memeriksa apakah data yang terurut berjarak sama terhadap data sebelumnya.

Masukan terdiri dari sekumpulan bilangan bulat yang diakhiri oleh bilangan negatif. Hanya bilangan non negatif saja yang disimpan kedalam array.

Keluaran terdiri dari dua baris. Baris pertama adalah isi dari array setelah dilakukan pengurutan, sedangkan baris ke dua adalah status jarak setiap bilangan yang ada didalam array. "Data berjarak x" atau "data berjarak tidak tetap".

Contoh masukan dan keluaran

No	Masukan	Keluaran
1	31 13 25 43 1 7 19 37 -5	1 7 13 19 25 31 37 43 Data berjarak 6
	4 40 14 8 26 1 38 2 32 -31	12 48 14 26 32 38 40 Data berjarak tidak tetap

Jawaban :

```
package main

import "fmt"

func insertionSort(arr []int) {
    n := len(arr)
    for i := 1; i < n; i++ {
```



```

        key := arr[i]
        j := i - 1
        for j >= 0 && arr[j] > key {
            arr[j+1] = arr[j]
            j--
        }
        arr[j+1] = key
    }
}

func main() {
    var arr []int

    for {
        var x int
        fmt.Scan(&x)
        if x < 0 {
            break
        }
        arr = append(arr, x)
    }

    insertionSort(arr)

    // Cetak array terurut
    for i, v := range arr {
        if i > 0 {
            fmt.Print(" ")
        }
        fmt.Print(v)
    }
    fmt.Println()

    // Cek jarak
    if len(arr) < 2 {
        fmt.Println("Data berjarak tidak tetap")
        return
    }

    jarak := arr[1] - arr[0]
    tetap := true
    for i := 2; i < len(arr); i++ {
        if arr[i]-arr[i-1] != jarak {
            tetap = false
            break
        }
    }
}

```

```

    }
}

if tetap {
    fmt.Printf("Data berjarak %d\n", jarak)
} else {
    fmt.Println("Data berjarak tidak tetap")
}
}

```

Output :

```

PROBLEMS  OUTPUT  TERMINAL  PORTS  DEBUG CONSOLE
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 12 - Modul 14\unguided3
31 13 25 43 1 7 19 37 -5
1 7 13 19 25 31 37 43
Data berjarak 6
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 12 - Modul 14\unguided3
4 40 14 8 26 1 38 2 32 -31
1 2 4 8 14 26 32 38 40
Data berjarak tidak tetap
PS D:\Praktikum\109082500162_Ais-Nurhanifah> 

```

Penjelasan :

Program ini membaca sekumpulan bilangan bulat yang diakhiri dengan bilangan **negatif** sebagai penanda berhenti. Hanya bilangan **non-negatif** yang disimpan ke dalam array. Setelah semua data terbaca, array diurutkan menggunakan algoritma **insertion sort** secara ascending. Berbeda dengan selection sort yang mencari elemen minimum, insertion sort bekerja dengan cara mengambil elemen ke-i sebagai key, lalu menggeser elemen-elemen sebelumnya yang lebih besar ke kanan satu per satu hingga ditemukan posisi yang tepat untuk key tersebut. Proses ini menyerupai cara manusia mengurutkan kartu remi, di mana setiap kartu baru disisipkan ke posisi yang tepat di antara kartu-kartu yang sudah terurut.

Setelah diurutkan, program memeriksa apakah selisih antara setiap dua elemen berurutan selalu sama atau tidak. Caranya adalah dengan menghitung selisih antara elemen pertama dan kedua sebagai **jarak acuan**, kemudian membandingkan selisih setiap pasangan elemen berikutnya dengan jarak acuan tersebut. Jika semua selisih sama, maka program mencetak "Data berjarak x" di mana x adalah nilai selisih yang konsisten. Namun jika ada satu saja selisih yang berbeda, program langsung mencetak "Data berjarak tidak tetap". Pengecekan ini berguna untuk mengetahui apakah data membentuk deret aritmatika.

4. Soal Unguided 4

Sebuah program perpustakaan digunakan untuk mengelola data buku di dalam suatu perpustakaan. Misalnya terdefinisi struct dan array seperti berikut ini:

```
const nMax : integer = 7919
type Buku = <
    id, judul, penulis, penerbit : string
    eksemplar, tahun, rating : integer >

type DaftarBuku = array [ 1..nMax] of Buku
Pustaka : DaftarBuku
nPustaka: integer
```

Masukan terdiri dari beberapa baris. Baris pertama adalah bilangan bulat N yang menyatakan banyaknya data buku yang ada di dalam perpustakaan. N baris berikutnya, masing-masingnya adalah data buku sesuai dengan atribut atau field pada struct. Baris terakhir adalah bilangan bulat yang menyatakan rating buku yang akan dicari.

Keluaran terdiri dari beberapa baris. Baris pertama adalah data buku terfavorit, baris kedua adalah lima judul buku dengan rating tertinggi, selanjutnya baris terakhir adalah data buku yang dicari sesuai rating yang diberikan pada masukan baris terakhir.

Lengkapi subprogram-subprogram dibawah ini, sesuai dengan I.S. dan F.S yang diberikan.

```
procedure DaftarkanBuku(in/out pustaka : DaftarBuku, n : integer)
{I.S. sejumlah n data buku telah siap para piranti masukan
 F.S. n berisi sebuah nilai, dan pustaka berisi sejumlah n data buku}

procedure CetakTerfavorit(in pustaka : DaftarBuku, in n : integer)
{I.S. array pustaka berisi n buah data buku dan belum terurut
 F.S. Tampilan data buku (judul, penulis, penerbit, tahun)
terfavorit, yaitu memiliki rating tertinggi}

procedure UrutBuku( in/out pustaka : DaftarBuku, n : integer )
{I.S. Array pustaka berisi n data buku
 F.S. Array pustaka terurut menurun/mengecil terhadap rating.
Catatan: Gunakan metoda Insertion sort}

procedure Cetak5Terbaru( in pustaka : DaftarBuku, n integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan 5 judul buku dengan rating tertinggi
 Catatan: Isi pustaka mungkin saja kurang dari 5}

procedure CariBuku(in pustaka : DaftarBuku, n : integer, r : integer )
{I.S. pustaka berisi n data buku yang sudah terurut menurut rating
 F.S. Laporan salah satu buku (judul, penulis, penerbit, tahun,
eksemplar, rating) dengan rating yang diberikan. Jika tidak ada buku
dengan rating yang ditanyakan, cukup tuliskan "Tidak ada buku dengan
rating seperti itu". Catatan: Gunakan pencarian biner/belah dua.}
```

Jawaban :

```
package main

import (
    "bufio"
    "fmt"
    "os"
)

const nMax = 7919

type Buku struct {
    id        string
    judul     string
    penulis   string
    penerbit  string
    eksemplar int
    tahun     int
    rating    int
}

type DaftarBuku [nMax]Buku

var (
    pustaka  DaftarBuku
    nPustaka int
    reader   = bufio.NewReader(os.Stdin)
)

func bacaLine() string {
    line, _ := reader.ReadString('\n')
    // trim \r\n
    for len(line) > 0 && (line[len(line)-1] == '\n' ||
line[len(line)-1] == '\r') {
        line = line[:len(line)-1]
    }
    return line
}

// DaftarkanBuku: membaca n data buku dari input
func DaftarkanBuku(pustaka *DaftarBuku, n int) {
    for i := 0; i < n; i++ {
        fmt.Print("ID Buku      : ")
        pustaka[i].id = bacaLine()
    }
}
```

```

        fmt.Print("Judul      : ")
        pustaka[i].judul = bacaLine()

        fmt.Print("Penulis     : ")
        pustaka[i].penulis = bacaLine()

        fmt.Print("Penerbit    : ")
        pustaka[i].penerbit = bacaLine()

        fmt.Print("Eksemplar   : ")
        fmt.Fscan(reader, &pustaka[i].eksemplar)
        reader.ReadString('\n')

        fmt.Print("Tahun Terbit : ")
        fmt.Fscan(reader, &pustaka[i].tahun)
        reader.ReadString('\n')

        fmt.Print("Rating      : ")
        fmt.Fscan(reader, &pustaka[i].rating)
        reader.ReadString('\n')

        fmt.Println()
    }
}

// CetakTerfavorit: cetak buku dengan rating tertinggi (belum
// diurutkan)
func CetakTerfavorit(pustaka DaftarBuku, n int) {
    if n == 0 {
        fmt.Println("Tidak ada data buku.")
        return
    }
    maxIdx := 0
    for i := 1; i < n; i++ {
        if pustaka[i].rating > pustaka[maxIdx].rating {
            maxIdx = i
        }
    }
    b := pustaka[maxIdx]
    fmt.Println("=== Buku Terfavorit ===")
    fmt.Println("Judul      :", b.judul)
    fmt.Println("Penulis     :", b.penulis)
    fmt.Println("Penerbit    :", b.penerbit)
    fmt.Println("Tahun      :", b.tahun)

```

```

        fmt.Println("Rating   :", b.rating)
    }

    // UrutBuku: insertion sort descending berdasarkan rating
    func UrutBuku(pustaka *DaftarBuku, n int) {
        for i := 1; i < n; i++ {
            key := pustaka[i]
            j := i - 1
            for j >= 0 && pustaka[j].rating < key.rating {
                pustaka[j+1] = pustaka[j]
                j--
            }
            pustaka[j+1] = key
        }
    }

    // Cetak5Terbaru: cetak 5 buku dengan rating tertinggi (sudah
    diurutkan)
    func Cetak5Terbaru(pustaka DaftarBuku, n int) {
        limit := 5
        if n < 5 {
            limit = n
        }
        fmt.Println("=== 5 Buku dengan Rating Tertinggi ===")
        for i := 0; i < limit; i++ {
            fmt.Printf("%d. %s (Rating: %d)\n", i+1, pustaka[i].judul,
pustaka[i].rating)
        }
    }

    // CariBuku: binary search berdasarkan rating (array sudah diurutkan
    descending)
    // Binary search pada array descending: cari dari kiri, nilai besar
    di kiri
    func CariBuku(pustaka DaftarBuku, n int, r int) {
        low := 0
        high := n - 1
        hasil := -1

        for low <= high {
            mid := (low + high) / 2
            if pustaka[mid].rating == r {
                hasil = mid
                break
            } else if pustaka[mid].rating > r {

```

```

        // nilai mid lebih besar, target ada di kanan (index
        lebih besar)
        low = mid + 1
    } else {
        // nilai mid lebih kecil, target ada di kiri (index lebih
        kecil)
        high = mid - 1
    }
}

if hasil == -1 {
    fmt.Println("Tidak ada buku dengan rating seperti itu")
} else {
    b := pustaka[hasil]
    fmt.Println("=== Data Buku Ditemukan ===")
    fmt.Println("Judul      :", b.judul)
    fmt.Println("Penulis     :", b.penulis)
    fmt.Println("Penerbit    :", b.penerbit)
    fmt.Println("Tahun       :", b.tahun)
    fmt.Println("Eksemplar   :", b.eksemplar)
    fmt.Println("Rating      :", b.rating)
}
}

func main() {
    fmt.Print("Masukkan jumlah buku: ")
    fmt.Fscan(reader, &nPustaka)
    reader.ReadString('\n')
    fmt.Println()

    // 1. Daftarkan buku
    DaftarkanBuku(&pustaka, nPustaka)

    // 2. Cetak buku terfavorit (sebelum diurutkan)
    CetakTerfavorit(pustaka, nPustaka)
    fmt.Println()

    // 3. Urutkan buku berdasarkan rating (descending)
    UrutBuku(&pustaka, nPustaka)

    // 4. Cetak 5 buku terbaru (rating tertinggi)
    Cetak5Terbaru(pustaka, nPustaka)
    fmt.Println()

    // 5. Cari buku berdasarkan rating

```

```

var ratingCari int
fmt.Print("Masukkan rating yang dicari: ")
fmt.Fscan(reader, &ratingCari)
CariBuku(pustaka, nPustaka, ratingCari)
}

```

Output :

```

PROBLEMS OUTPUT TERMINAL PORTS DEBUG CONSOLE
PS D:\Praktikum\109082500162_Ais-Nurhanifah> go run "d:\Praktikum\109082500162_Ais-Nurhanifah\Week 12 - Modul 14\unguided4.go"
Masukkan jumlah buku: 3

ID Buku      : B01
Judul        : Struktur Data Go
Penulis      : Ahmad Wijaya
Penerbit     : Infrmatix Press
Eksemplar    : 10
Tahun Terbit : 2024
Rating       : 5

ID Buku      : B02
Judul        : Algoritma Dasar
Penulis      : Budi Santoso
Penerbit     : Tekno Media
Eksemplar    : 15
Tahun Terbit : 2023
Rating       : 3

ID Buku      : B03
Judul        : Pemrograman Web
Penulis      : Citra Lestari
Penerbit     : Edukasi Digital
Eksemplar    : 8
Tahun Terbit : 2025
Rating       : 4

=== Buku Terfavorit ===
Judul      : Struktur Data Go
Penulis    : Ahmad Wijaya
Penerbit   : Infrmatix Press
Tahun     : 2024
Rating    : 5

=== 5 Buku dengan Rating Tertinggi ===
1. Struktur Data Go (Rating: 5)
2. Pemrograman Web (Rating: 4)
3. Algoritma Dasar (Rating: 3)

Masukkan rating yang dicari: 4
=== Data Buku Ditemukan ===
Judul      : Pemrograman Web
Penulis    : Citra Lestari
Penerbit   : Edukasi Digital
Tahun     : 2025
Eksemplar  : 8
Rating     : 4
PS D:\Praktikum\109082500162_Ais-Nurhanifah>

```

Penjelasan :

Program **perpustakaan** adalah program yang digunakan untuk mengelola data buku di dalam suatu perpustakaan menggunakan struct Buku yang memiliki field id, judul, penulis, penerbit bertipe string serta eksemplar, tahun, dan rating bertipe integer. Program ini terdiri dari beberapa subprogram yaitu DaftarkanBuku untuk menginput data buku, CetakTerfavorit untuk menampilkan

buku dengan rating tertinggi sebelum diurutkan, UrutBuku untuk mengurutkan data buku berdasarkan rating secara descending menggunakan **insertion sort**, Cetak5Terbaru untuk menampilkan 5 buku dengan rating tertinggi setelah diurutkan, dan CariBuku untuk mencari buku berdasarkan rating menggunakan **binary search**.

Algoritma **insertion sort descending** pada UrutBuku bekerja dengan mengambil elemen ke-*i* sebagai key, lalu menggeser elemen-elemen sebelumnya yang **lebih kecil** dari key ke kanan satu per satu hingga ditemukan posisi yang tepat. Sementara itu, **binary search** pada CariBuku memanfaatkan fakta bahwa array sudah terurut descending, sehingga jika nilai tengah (*mid*) lebih besar dari target maka pencarian dilanjutkan ke bagian kanan (*index* lebih besar), dan jika lebih kecil maka ke bagian kiri (*index* lebih kecil). Kombinasi kedua algoritma ini membuat program perpustakaan menjadi efisien dalam pengurutan maupun pencarian data buku.

D. Kesimpulan

Berdasarkan hasil praktikum yang telah dilaksanakan, dapat disimpulkan bahwa algoritma pengurutan (*sorting*) memiliki peran yang sangat vital dalam menstrukturkan data agar program dapat berjalan lebih optimal. Pemilihan algoritma yang tepat seperti *Selection Sort* dan *Insertion Sort* sangat bergantung pada kebutuhan efisiensi kode serta karakteristik problem linear di dalam bahasa Go.

Algoritma *Selection Sort* terbukti efektif dalam melakukan pengurutan dengan memanfaatkan konsep pencarian nilai ekstrem di setiap iterasinya dengan jumlah operasi pertukaran (*swap*) yang relatif minim. Di sisi lain, algoritma *Insertion Sort* menawarkan mekanisme pengurutan yang lebih adaptif dan dinamis dengan cara menyisipkan elemen data satu per satu ke dalam posisi yang benar melalui proses pergeseran (*shifting*). Namun, karena memiliki kompleksitas waktu $O(n^2)$ akibat penggunaan dua perulangan bersarang, kedua algoritma ini kurang direkomendasikan untuk menangani sekumpulan data dalam skala yang sangat besar.

Secara keseluruhan, pemahaman terhadap penggunaan *pointer* dan manipulasi *array of struct* menjadi kunci utama keberhasilan implementasi kedua algoritma pengurutan tersebut dalam menyelesaikan studi kasus nyata. Implementasi pengurutan pada objek *struct* mampu menjamin bahwa semua atribut data yang saling terikat tidak akan terpisah atau tertukar dengan record milik entitas lain saat proses pengurutan berlangsung. Penguasaan metode pengurutan menaik (*ascending*) maupun menurun (*descending*) pada berbagai tipe data ini menjadi fondasi penting bagi pemrogram sebelum melangkah ke struktur data yang lebih kompleks dan algoritma pencarian lanjutan.